

PROGRAMMAZIONE II

SINTASSI BASE DI JAVA

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

COSTRUTTI SEMPLICI IN JAVA



VARIABILI

Una variabile è una locazione di memoria con un tipo associato (primitivo o riferimento)

Una variabile contiene sempre dati compatibili con il suo tipo

Il valore di una variabile può cambiare a causa di assegnazioni o incrementi

Tutti i tipi hanno un valore di default, usato all'inizializzazione della variabile

- ▶ 0 per `int`, `null` per i riferimenti, `false` per `boolean`, . . .
- ▶ L'inizializzazione automatica viene eseguita solo sui campi di classe (fare attenzione)

VALORI DI DEFAULT

- ✕ Nessun supporto nativo per i numeri senza segno

Tipo	Intervallo di valori	Valore di default
byte	Interi da -2^7 a $2^7 - 1$	0
short	Interi da -2^{15} a $2^{15} - 1$	0
int	Interi da -2^{31} a $2^{31} - 1$	0
long	Interi da -2^{63} a $2^{63} - 1$	0L
float	Razionali con 6 - 7 cifre decimali	0.0f
double	Razionali con 15 - 16 cifre decimali	0.0
boolean	Valori di verità	false
char	Singoli valori ASCII	'\u0000'
String (qualsiasi oggetto)	Dati non primitivi	null

DICHIARAZIONE DI VARIABILI

Le variabili devono essere dichiarate prima del loro uso

```
int i;    0    int i = 5;
```

Le dichiarazioni possono apparire ovunque nel codice

- All'interno o all'esterno dei metodi di classe, dentro rami o corpi di cicli, . . .

Dichiarazione e inizializzazione possono essere effettuate in momenti diversi

- Gli oggetti richiedono un'inizializzazione esplicita prima dell'uso, per evitare errori di tipo `NullPointerException`

COSTANTI

Una variabile può essere dichiarata come costante usando le parole chiave `final static`

```
final static int I = 5;
```

Una costante deve essere inizializzata e il suo valore non può essere modificato successivamente

```
int j;
final static int I; // errore di compilazione
final static int I = 6;

j = 5;
I = 3; // errore di compilazione
```

1
2
3
4
5
6

IDENTIFICATORI

Un identificatore è un nome usato per riferirsi a variabili, metodi, classi, costanti, . . .

Tecnicamente, una sequenza illimitata di caratteri in [a-z], [A-Z], [0-9], più _ e \$

Convenzioni di denominazione:

- ▶ Usare la convenzione camel case per nomi composti senza trattini bassi
- ▶ I nomi delle classi devono avere l'iniziale maiuscola, es. `AerialDrone`
- ▶ Metodi, oggetti, campi, variabili di metodo devono iniziare con la minuscola, es. `getCoordinates()`, `myCar`
- ▶ Le costanti devono essere tutte maiuscole con trattini bassi, es. `PI_CONST = 3.14`

IDENTIFICATORI

Una parola chiave è una sequenza riservata di caratteri, non consentita come identificatore

abstract	default	if	private	throw
boolean	do	implements	protected	throws
break	double	import	public	transient
byte	else	instanceof	return	try
case	extends	int	short	void
catch	final	interface	static	volatile
char	finally	long	super	while
class	float	native	switch	
const	for	new	synchronized	
continue	goto	package	this	

CARATTERI SPECIALI

Codifica di caratteri grafici da usare nella formattazione dell'output

```
\b  /* \u0008: backspace BS */
\t  /* \u0009: tabulazione orizzontale HT */
\n  /* \u000a: avanzamento riga LF */
\f  /* \u000c: avanzamento pagina FF */
\r  /* \u000d: ritorno a capo CR */
\"  /* \u0022: virgolette doppie */
\'  /* \u0027: virgoletta singola */
\\  /* \u005c: barra inversa (backslash) */
```

COMMENTI

Commenti su riga singola `// ...`

- ▶ Ciò che segue `//` viene ignorato

Commenti su più righe `/* ... */`

- ▶ Ciò che è racchiuso tra `/*` e `*/` viene ignorato

Commenti di documentazione `/** ... */`

- ▶ Ciò che è racchiuso tra `/**` e `*/` viene ignorato
- ▶ Ma può essere elaborato da javadoc per produrre documentazione

ESPRESSIONI

Un'espressione è un termine che viene infine valutato a un valore

$$\text{eval}(4) = 4 \quad \text{eval}(x + 1) = \text{eval}(x) + \text{eval}(1) = 3 + 1 = 4$$

Pertanto, le espressioni hanno un tipo associato

- ▶ 4 ha tipo `int`
- ▶ Dato `int` x, allora x ha tipo `int`
- ▶ Dato `int` x, allora x + 1.0 ha tipo `double`

// cast implicito sicuro

ESPRESSIONI (CONT.)

Le espressioni possono avere effetti collaterali

```
int i = 1; int j = (i=2) * i;
```

Qual è il valore di i e j dopo l'esecuzione?

- ▶ i assumerà 2 mentre j assumerà 4
- ▶ Le espressioni vengono valutate da sinistra a destra

Dato `int t = 3`, qual è la valutazione di `t + (t=2)`?

$$\text{eval}(t + (t=2)) = \text{eval}(t) + \text{eval}(t=2) = 3 + 2 = 5$$

NOMENCLATURA

A seconda del loro tipo, le espressioni sono chiamate

- ▶ Espressioni booleane, se valutano a **boolean** // o condizioni
- ▶ Espressioni aritmetiche, se valutano a qualsiasi tipo primitivo eccetto **boolean**
- ▶ Espressioni riferimento, se valutano a qualsiasi oggetto

Useremo la seguente meta-sintassi per riferirci alle espressioni

<i>expr</i>	Espressione generica
<i>Bexpr</i>	Espressione booleana
<i>Aexpr</i>	Espressione aritmetica
<i>Rexpr</i>	Espressione riferimento

OPERATORI

Usati nella costruzione di espressioni per eseguire calcoli a partire da operandi

- ▶ Unario $\ominus Aexpr$ // es. negazione
- ▶ Binario $expr1 \otimes expr2$ // es. moltiplicazione
- ▶ Ternario $Bexpr ? expr1 : expr2$ // condizionale inline

Per esempio, l'espressione $x < 0 ? -1 : 1$

- ▶ Valuta a -1 quando x vale -5
- ▶ Valuta a 1 quando x vale 8
- ▶ $expr1$ ed $expr2$ possono avere tipo diverso

OPERATORI ARITMETICI BINARI

Sottrazione $Aexpr1 - Aexpr2$

Divisione $Aexpr1 / Aexpr2$

Addizione $Aexpr1 + Aexpr2$

Moltiplicazione $Aexpr1 * Aexpr2$

Modulo $Aexpr1 \% Aexpr2$

Concatenazione $Rexpr1 + Rexpr2$

Overloading degli operatori

- ▶ `int + int` produrrà un `int`
- ▶ `double + double` produrrà un `double`
- ▶ `String + String` produrrà una `String`

OPERATORI ARITMETICI UNARI

++ incrementa di 1

-- decrementa di 1

Negazione -expr

Post-incremento

// analogo per il decremento

► Se x vale 4, allora x++ valuta a 4 e successivamente x viene incrementato di 1

Pre-incremento

// analogo per il decremento

► Se x vale 4, allora ++x valuta a 5 poiché x viene prima incrementato di 1

Dato `int t = 3`, qual è il valore di t dopo `t = -t++ - ++t`?

$$\text{eval}(-t++ - ++t) = (-\text{eval}(t++)) - \text{eval}(++t) = (-3) - 5 = -3 - 5 = -8$$

OPERATORI RELAZIONALI

Usati per confrontare espressioni aritmetiche per prendere decisioni di logica di business

$Aexpr_1 > Aexpr_2$	Valuta a true se $Aexpr_1$ è maggiore di $Aexpr_2$
$Aexpr_1 \geq Aexpr_2$	Valuta a true se $Aexpr_1$ è maggiore o uguale ad $Aexpr_2$
$Aexpr_1 < Aexpr_2$	Valuta a true se $Aexpr_1$ è minore di $Aexpr_2$
$Aexpr_1 \leq Aexpr_2$	Valuta a true se $Aexpr_1$ è minore o uguale ad $Aexpr_2$
$Aexpr_1 == Aexpr_2$	Valuta a true se $Aexpr_1$ è uguale ad $Aexpr_2$
$Aexpr_1 != Aexpr_2$	Valuta a true se $Aexpr_1$ è diverso da $Aexpr_2$

Valutano a boolean

OPERATORI LOGICI

Usati per confrontare espressioni booleane per prendere decisioni di logica di business

$! Bexpr$	Valuta a true se $Bexpr$ è false
$Bexpr_1 \& Bexpr_2$	Valuta a true se $Bexpr_1$ è true e $Bexpr_2$ è true
$Bexpr_1 Bexpr_2$	Valuta a true se $Bexpr_1$ è true o $Bexpr_2$ è true
$Bexpr_1 \&\& Bexpr_2$	Valuta a true se $Bexpr_1$ è true e $Bexpr_2$ è true
$Bexpr_1 Bexpr_2$	Valuta a true se $Bexpr_1$ è true o $Bexpr_2$ è true

SC
SC

Valutazione a corto circuito (Short-Circuit)

- ▶ In $Bexpr_1 \&\& Bexpr_2$, se $Bexpr_1$ è false allora $Bexpr_2$ non viene valutata
- ▶ In $Bexpr_1 || Bexpr_2$, se $Bexpr_1$ è true allora $Bexpr_2$ non viene valutata
- ▶ Efficiente, ma può portare a comportamenti imprevisti se $Bexpr_2$ ha effetti collaterali

OPERATORI DI ASSEGNAZIONE

Usati per inizializzare esplicitamente una variabile, o per cambiarne il valore

<i>var = expr</i>	Valuta <i>expr</i> e assegna il valore a <i>var</i>
<i>var += expr</i>	Equivalente a <i>var = var + expr</i>
<i>var -= expr</i>	Equivalente a <i>var = var - expr</i>
<i>var *= expr</i>	Equivalente a <i>var = var * expr</i>
<i>var /= expr</i>	Equivalente a <i>var = var / expr</i>
<i>var %= expr</i>	Equivalente a <i>var = var % expr</i>

Per esempio, dato `int x = 4`

- ▶ `x -= 3` assegnerà a `x` il valore 1
- ▶ `x += x - 7` assegnerà a `x` il valore 1, equivalente a `x = x + (x - 7)`

COSTRUTTI DI CONTROLLO DEL FLUSSO IN JAVA



BLOCCHI DI CODICE

Elenco di istruzioni racchiuse tra parentesi graffe { ... }

- ▶ Ogni blocco inizia un nuovo scope per le variabili
- ▶ Le variabili dichiarate in un blocco sono disponibili solo in quel blocco

```
{ 1
  int j = 4; 2
  j++; // j vale 5 qui 3
} 4
{ 5
  int j = 7; 6
  --j; // j vale 6 qui 7
} 8
// j non disponibile qui 9
```

BLOCCHI DI CODICE (CONT.)

Un blocco di codice può esistere autonomamente o appartenere a condizionali e cicli

- Un blocco di codice eredita lo scope dei blocchi a cui appartiene

Quando i blocchi di codice contengono una singola istruzione le graffe possono essere omesse

`{ x = 2; }` è equivalente a `x = 2;`

ISTRUZIONI CONDIZIONALI

Istruzioni che consentono di selezionare dinamicamente il codice da eseguire

if (Bexpr) block e if (Bexpr) block else block

<code>if (i < 0) j++;</code>	1
<code>else { j--; }</code>	2
<code>// il valore di j qui dipende da i</code>	3

 0 e null non sono equivalenti a false

ISTRUZIONI CONDIZIONALI (CONT.)

Istruzioni case, semplificano le catene if-else

```
switch (Aexpr) {
    case Aexpr1: block1 break;
    ...
    case Aexprn: blockn break;
    default: block break;
}
```

```
switch (month) {
    case 4:
    case 6:
    case 9:
    case 11:
        days = 30;
        break;
    case 2:
        days = 28;
        break;
    default:
        days = 31;
        break;
}
```

Consentiti solo **byte**, **short**, **char**, **int** e **long** (String ed **enum** aggiunti da Java 7)

ISTRUZIONI DI CICLO

Istruzioni che consentono di ripetere dinamicamente l'esecuzione del codice

`while (Bexpr) block` e `do (Bexpr) block while (Bexpr);`

```
int s = 0;           1
int i = 0;           2
while (i <= 9) {      3
    s += i;           4
    i++;              5
}                    6
// s vale 45 qui     7
```

```
int s = 0;           1
int i = 0;           2
do {                 3
    s += i;           4
    i++;              5
} while (i <= 9);     6
// s vale 45 qui     7
```

ISTRUZIONI DI CICLO (CONT.)

Gauss at his 7

$$\sum_{k=0}^n k = \frac{n(n+1)}{2}; (k, n) \in \mathbb{N}^2$$



Me at my 19

$$77 + 33 = 100$$



CICLI CONTROLLATI

Cicli semplificati per iterazioni controllate

for (*init*; *Bexpr*; *inc/dec*) { *block* }

```
int s = 0;
for (int i = 0; i <= 9; i++) {
    s += i;
}
// s vale 45 qui
```

Codice più semplice

```
int s = 0;
int i, j;
for (i = 0, j = 9; i <= 9 && j >= 0; i++, j--)
    s += i + j;
// s vale 90 qui
```

Inizializzazioni multiple
 Inc/dec multipli

ISTRUZIONI DI CONTROLLO DIRETTO DEL FLUSSO

Istruzioni che scavalcano il controllo logico nei condizionali e nei cicli

- `break;` Salto fuori da un ciclo (termina il ciclo)
- `continue;` Salto alla successiva iterazione del ciclo
- `return expr;` Uscita da un metodo con restituzione del valore di *expr*

```
int s = 0;           1
for (int i = 0; i <= 9; i++) { 2
    if (i == 4) break;    3
    s += i;              4
                        5
// s vale 6 invece di 45 6
```

```
int s = 0;           1
for (int i = 0; i <= 9; i++) { 2
    if (i == 4) continue; 3
    s += i;          4
                    5
// s vale 41 invece di 45 6
```

DOMANDE E RISPOSTE

